Chapter 16A: Online Learning and Testing

In this next example we are going to learn something about the application design process. When you start your IT career someone else is typically going to do the design and you are going to be asked to realize it. Eventually however, they are going to ask you to do some of your own design. So how does one go about that monumental task given you probably don't know a single thing about the business in which you are being asked to design?

When I joined my last group I knew absolutely nothing about human performance testing. At the time the compay was doing only paper and pencil testing where you filled in the little dots on a Scantron sheet and then waited weeks to get your results back. Their customers were exclusively professional organizations who would schedule a large banquet hall a couple of times each year to administer their tests. Our company wrote and scored those tests in conjunction with subject matter experts provided by the customer.

I was hired to bring those tests online such that a network of testing sites could be established and testing candidates could take their exams whenever and wherever they wanted. If possible, the candidates would also get their results back immediately upon completing the test.

Twelve years later, we had exceeded expectations. We operated a global network of more than 2500 testing centers where candidates could schedule and sit for high-stakes licensure and certification exams. We also delivered lower-stakes practice tests and continuing education programs, sharing the fees with the sponsoring customer.

During that 12 year period the organization grew 550% in revenue and 2000% in EBITDA. If those terms are unfamiliar, learn them well: revenue and EBITDA are essential meausures of a business's health and performance.

So how did we do it?

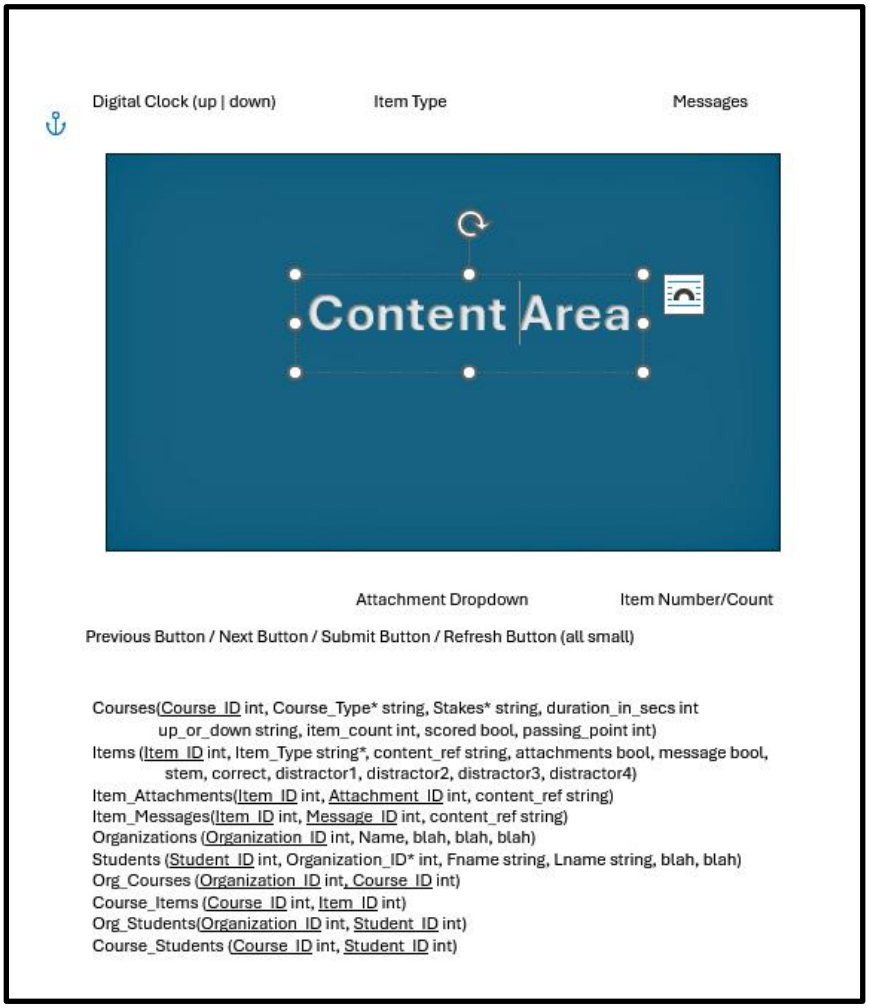
I started by talking with the subject matter experts. In the testing business those people are called Psychometricians, Ph.Ds. in human performance measurement. The good news was my company had a bunch of them, and they were all more than willing to share their expertise.

Next, I spoke with the sales and marketing team. If the sky were the limit what else could you be selling to our current and potential future customers?

With that knowledge in hand, I started sketching out what might be a plausible system. My initial rough design looked something like that shown here. (Figure 16A.1)

When working with subject matter experts, I found that they were excellent at explaining how things currently worked, but far less effective at envisioning how they might work into the future. My initial design therefore focused on multiple-choice questions organized into high-stakes exams, while recognizing that the definition of "content" could expand over time.

Figure 16A.1: Rough-Cut Initial Design

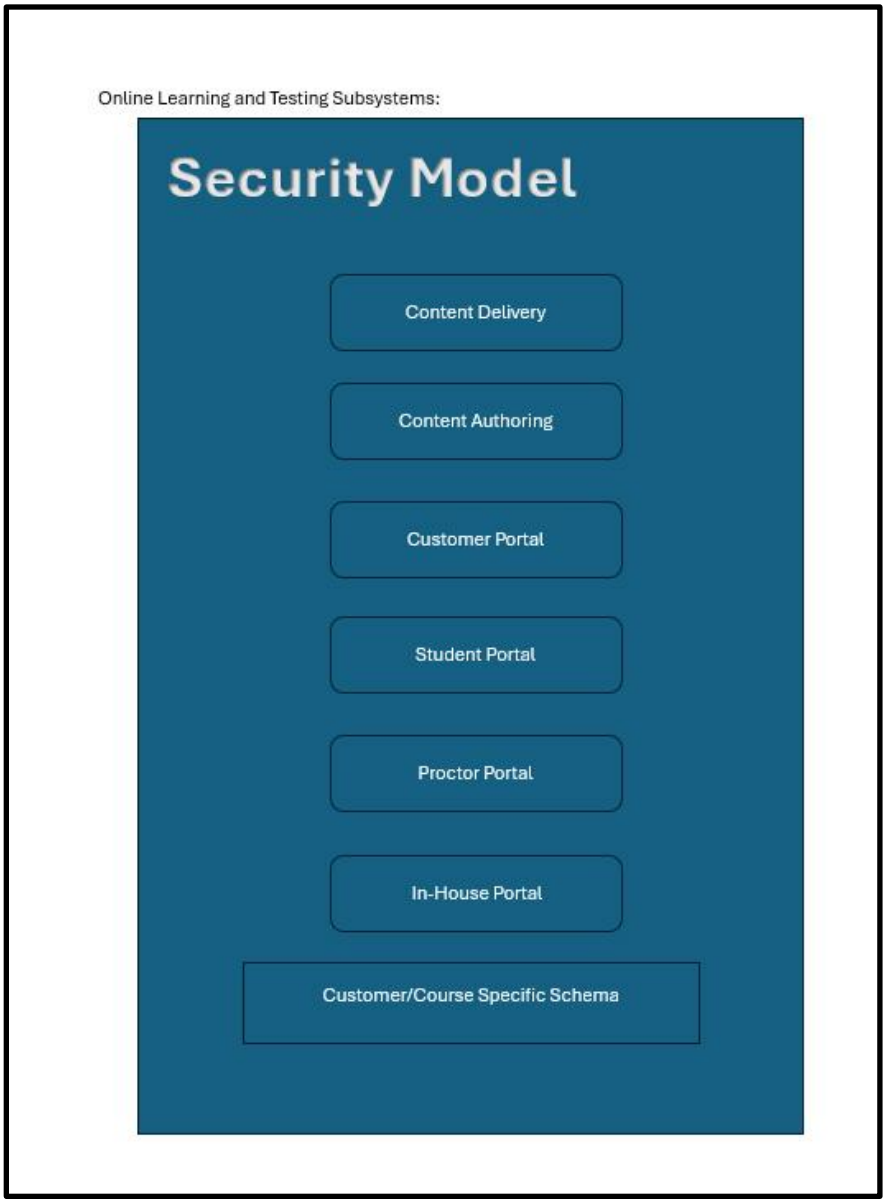


Some explanation of this design is probably warranted. First, the top part of the image is my rough idea of how a page might look that could display a multiple-choice question or any other type of content, e.g. e-learning, true-false question, video, etc., along with its associated instructions (messages) and support content (attachments). The timer could be used in either count up or count down mode per the customer's request. The timer could be visible for either high-stakes (certification or licensure) or low-stakes practice tests or made invisible for e-learning, again per the customer's determination.

The bottom part of the image is my initial cut at some database tables where the underlined attributes represent the unique identifier of each record. If this were the real deal this would all be done on a whiteboard which then gets printed out and distributed to the team so we could start to realize the solution knowing it would certainly change over time.

Before anybody on the team even opens Visual Studio we make certain everyone understands the overall vision. The design shown here attempts to break the solution down into its principle components. (Figure 16A.2)

Figure 16A.2: Solution Components



The content delivery subsystem is pretty self-explanatory. We get a learner or test taker (which we referred to as a student even through many of them were seasoned professionals taking a licensure or certification exam) in front of a terminal and present them with some content which is either proctored and scored as a high-stakes exam, not proctored but scored as a practice test or was a simple page turner, i.e. e-learning. The content delivery subsystem would be multilingual.

The content authoring subsystem would be used by the Psychometricians and e-learning folks to author and organize their content. We all agreed to call that collection of content a course knowing very well that it might also be a test. That collection of content to be displayed on a single page we called an item and each item had an item type, e.g. multiple-choice question, true false question, free response question, matching question, e-learning video, e-learning text, etc. Each item type would have a distinct setup page, a distinct delivery page, and a distinct recovery page.

The customer portal would allow customers to see how many students were going through the system and their aggregate results. Individual results were to be released only to the student. To keep things easy that subsystem would employ a large number of infographics.

The student portal would be used to sign up for a course, pay for that course, and if appropriate schedule a slot at a testing site. The subsystem would be multilingual, but all e-commerce transactions would be done in US dollars.

The proctor portal would allow exam proctors to schedule which hours were available for testing at their site. This subsystem would also be multilingual.

The in-house portal would allow customer support representatives to help distressed students and proctors. These same individuals would also use this subsystem to set up security and create and drop testing sites and proctors.

All this would be realized on customer-by-course specific database schemas to give our customers confidence that their information would never be intermixed with that of other customers or courses.

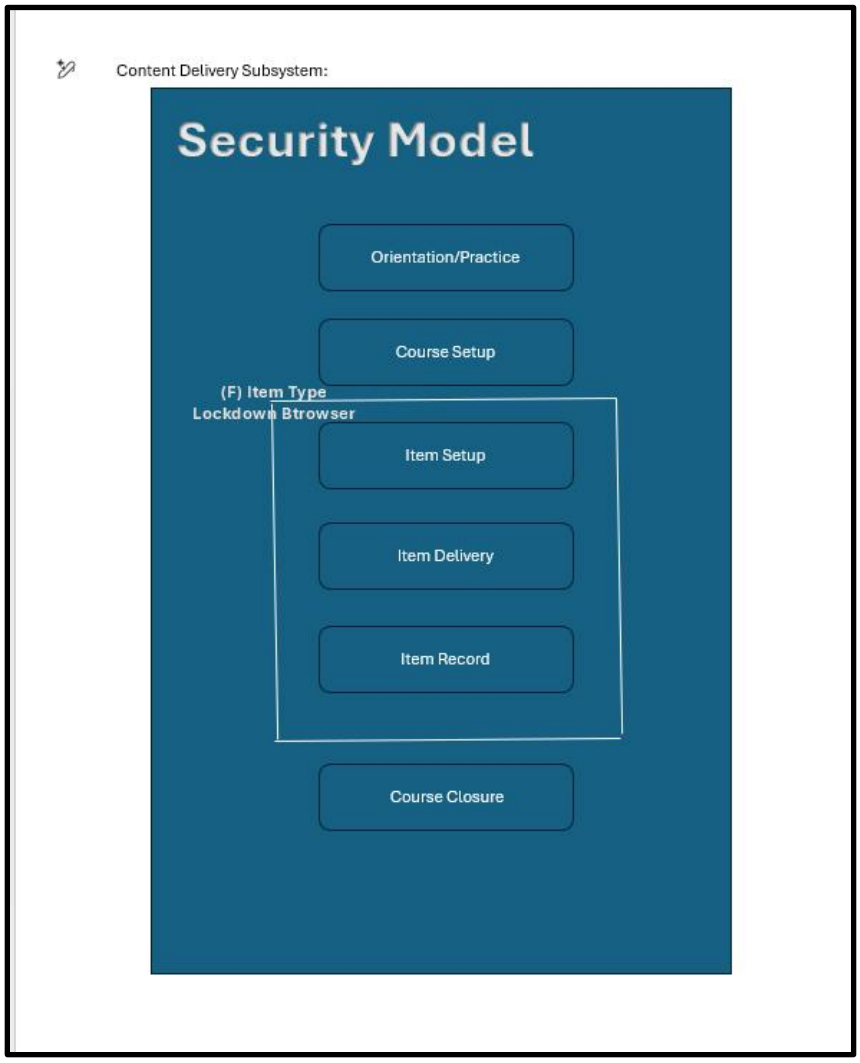
All subsystems would employ a role-based security model. Within a role-based security model, named roles are created and assigned to a set of system functions. Any user who is assigned to that role may then perform those functions thus setting up a security matrix within the solution.

Next came a set of rules of engagement. Large teams need to all play in the same sandbox or things quickly fall apart. I was never one for a whole lot of team rules, so things were pretty loose, but they did include the following:

- Global variables and classes were not only allowed but encouraged. More on global variables when we look at some code.
- Each team would utilize a unique namespace.
- Every team would have a team leader who would assign daily tasks and monitor progress.
- Everybody would initially be assigned to a team, but exchanges could happen if approved by the impacted team leaders.
- Teams would meet individually first thing every morning to discuss yesterday's successes and challenges. Any team member could seek advice and counsel during their morning team meeting without judgement or retribution.
- Team leaders would meet daily at 6pm to discuss successes, challenges and team member dynamics.
- There would be one team with only a single member, our most senior developer. He would be responsible for the creation of the lockdown browser and would only attend the end-of-day meetings once per week on Friday to provide an update and solicit input and advice.

Here is the deal, this solution probably had a couple hundred screens and tens of thousands of lines of code. Therefore, for the sake of brevity, we will only explore the content delivery subsystem shown here. (Figure 16A.3) Again, please note how we haven't written a single line of code yet. Think, then type was the mantra. Get the overall design agreed upon by everyone and only then do we create teams, assign components and start writing code.

Figure 16A.3: Content Delivery Subsystem



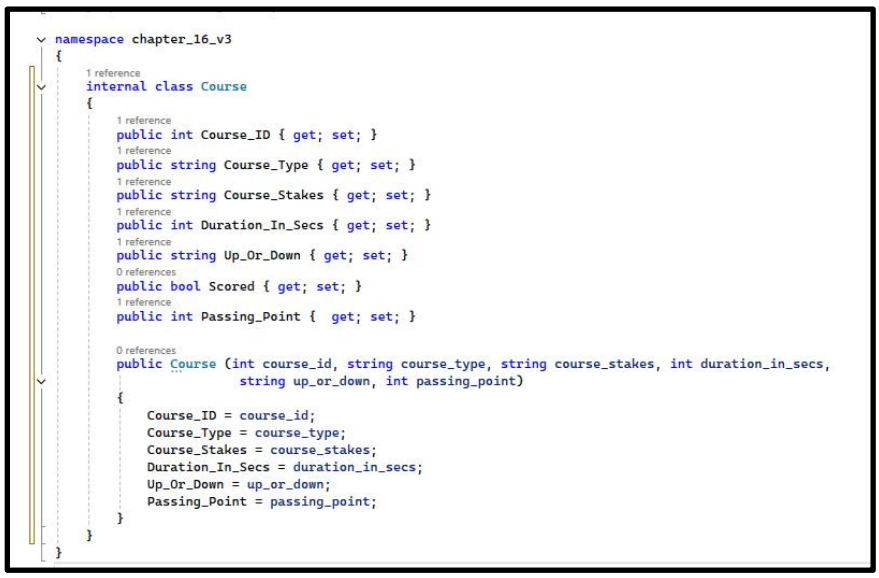
Our company was going to be the first one in the world to deliver high-stakes examinations over the internet. Do you think the competition had a bunch of negative comments when they heard what we were attempting? “How are you going to maintain security?” “What about language barriers?” “I simply don’t think it is possible….” Blah, blah, blah, blah ….

Well, I knew from the beginning that the toughest part of this solution was going to be getting around the internet security issues. Let’s say someone is taking a licensing exam to practice as a physician and is able to go search the web for answers, not acceptable. The solution was pretty straightforward, we needed what we termed a lockdown browser. A browser which could display all forms of HTML without allowing the user to surf the web. The problem was no such browser existed, we had to invent it. The solution meant reverse-engineering one of the currently available browsers and rebuilding it to disallow any of the regular buttons and/or key stroke combinations. Absolutely no escaping to another URL. Yes, six months later my most senior programmer had the solution working and we even got it patented.

Okay, finally time to open up Visual Studio and start looking at some code. My new project was called Chapter 16 Online Learning and Testing. I set the size of my home form to 820 by 900 and its Name and Text properties to Content. As is common practice, I set those properties in the Form Load event handler.

This figure shows my first cut at a Course class definition, we will instantiate that guy first. (Figure 16A.4) Please note how I have created a parallel parameterized constructor so other programmers immediately know what properties this guy requires when they instantiate the class. Also please note that all parameters are required and there are no defaults. As with our previous example, this class definition went into a separate file to minimize confusion.

Figure 16A.4: Course Class Definition



```
namespace chapter_16_v3
{
    1 reference
    internal class Course
    {
        1 reference
        public int Course_ID { get; set; }
        1 reference
        public string Course_Type { get; set; }
        1 reference
        public string Course_Stakes { get; set; }
        1 reference
        public int Duration_In_Secs { get; set; }
        1 reference
        public string Up_Or_Down { get; set; }
        0 references
        public bool Scored { get; set; }
        1 reference
        public int Passing_Point {  get; set; }

        0 references
        public Course (int course_id, string course_type, string course_stakes, int duration_in_secs,
                       string up_or_down, int passing_point)
        {
            Course_ID = course_id;
            Course_Type = course_type;
            Course_Stakes = course_stakes;
            Duration_In_Secs = duration_in_secs;
            Up_Or_Down = up_or_down;
            Passing_Point = passing_point;
        }
    }
}
```

Normally, a student would open a customer-specific portal, select a course, and retrieve the appropriate information from a database. Because we have not yet covered databases, I will instead use global variables at the top of the form class definition. The figure also shows the timer setup routine. (Figure 16A.5)

Academics often criticize global variables as a quick-and-dirty solution. In practice, however, professional programmers may use them when they wish to avoid passing large complex classes between routines.

By the way, I added the Digital_Clock and timer1 objects via the design screen so I knew their Control collection index. We will see in a moment why this was important.

Figure 16A.5: Global Variables

```
3 references
public partial class Form1 : Form
{
    string up_or_down, stakes, type;
    int duration_in_secs, passing_point;
    string time_in_minutes, time_in_seconds;
    1 reference
    public Form1()
    {
        InitializeComponent();
    }

    1 reference
    private void Form1_Load(object sender, EventArgs e)
    {
        this.Name = "Content";
        this.Size = new System.Drawing.Size(820, 900);
        this.Text = "Content";
        Digital_Clock.Size = new Size(150, 40);
        Digital_Clock.Location = new Point(670, 20);
        timer1.Interval = 1000;

        up_or_down = "Up";
        stakes = "Low";
        type = "MC";
        duration_in_secs = 0;  // not timed
        passing_point = 80;

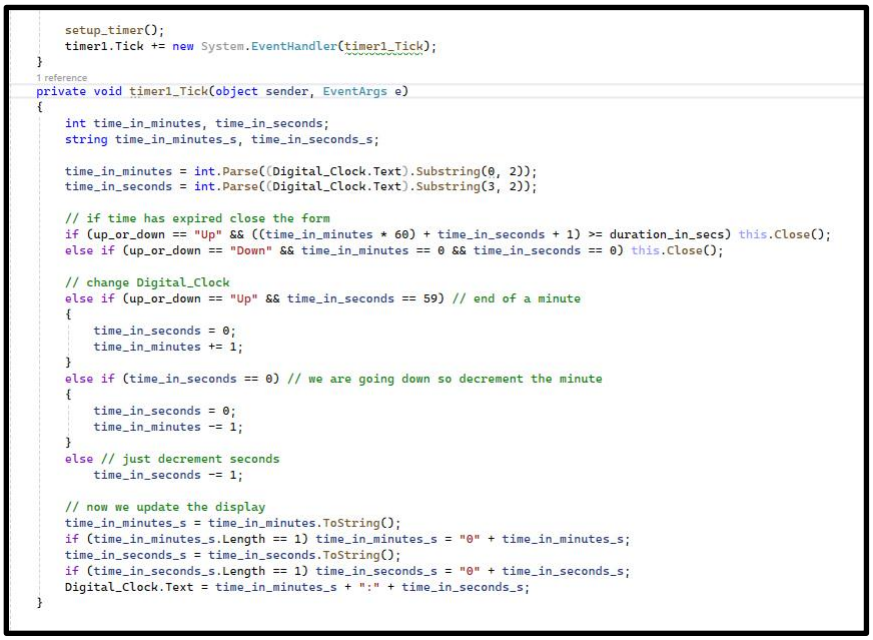
        setup_timer();
    }

    1 reference
    private void setup_timer()
    {
        time_in_minutes = (duration_in_secs / 60).ToString();
        time_in_seconds = (duration_in_secs % 60).ToString();
        if (time_in_seconds == "0") time_in_seconds="00";

        Digital_Clock.Text = time_in_minutes + ":" + time_in_seconds;
        Digital_Clock.Visible = true;
        timer1.Start();
    }
}
```

Next, it was time to write that Tick event handler for our timer. Not much going on in this routine other than the conversion between string and int and then back again. (Figure 16A.6)

Figure 16A.6: timer1_Tick



```
    setup_timer();
    timer1.Tick += new System.EventHandler(timer1_Tick);
}
1 reference
private void timer1_Tick(object sender, EventArgs e)
{
    int time_in_minutes, time_in_seconds;
    string time_in_minutes_s, time_in_seconds_s;

    time_in_minutes = int.Parse((Digital_Clock.Text).Substring(0, 2));
    time_in_seconds = int.Parse((Digital_Clock.Text).Substring(3, 2));

    // if time has expired close the form
    if (up_or_down == "Up" && ((time_in_minutes * 60) + time_in_seconds + 1) >= duration_in_secs) this.Close();
    else if (up_or_down == "Down" && time_in_minutes == 0 && time_in_seconds == 0) this.Close();

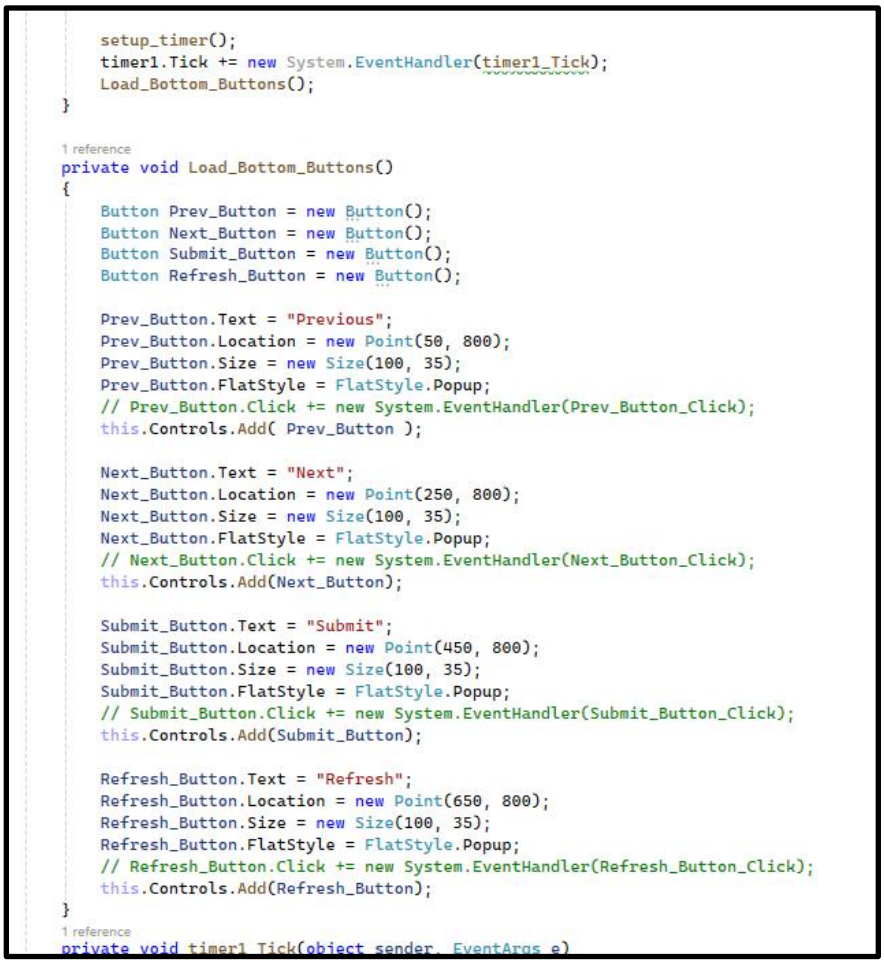
    // change Digital_Clock
    else if (up_or_down == "Up" && time_in_seconds == 59) // end of a minute
    {
        time_in_seconds = 0;
        time_in_minutes += 1;
    }
    else if (time_in_seconds == 0) // we are going down so decrement the minute
    {
        time_in_seconds = 0;
        time_in_minutes -= 1;
    }
    else // just decrement seconds
        time_in_seconds -= 1;

    // now we update the display
    time_in_minutes_s = time_in_minutes.ToString();
    if (time_in_minutes_s.Length == 1) time_in_minutes_s = "0" + time_in_minutes_s;
    time_in_seconds_s = time_in_seconds.ToString();
    if (time_in_seconds_s.Length == 1) time_in_seconds_s = "0" + time_in_seconds_s;
    Digital_Clock.Text = time_in_minutes_s + ":" + time_in_seconds_s;
}
```

I next chose to add those buttons across the bottom. We of course could have done this via the design window, but I chose to add those objects to the form in code. My Load_Bottom_Buttons routine is shown here. (Figure 16A.7) Please note how I added each button to the Controls collection of our form and also how we attached the associated click event. We will uncomment those click event handlers when we write the associated code in our next lesson.

By the way, I am going to discontinue providing those tabular specifications at this point. Every page in this book costs you extra money so please just read the code for the appropriate property values.

Figure 16A.7: Load_Bottom_Buttons



```
        setup_timer();
        timer1.Tick += new System.EventHandler(timer1_Tick);
        Load_Bottom_Buttons();
    }

    1 reference
    private void Load_Bottom_Buttons()
    {
        Button Prev_Button = new Button();
        Button Next_Button = new Button();
        Button Submit_Button = new Button();
        Button Refresh_Button = new Button();

        Prev_Button.Text = "Previous";
        Prev_Button.Location = new Point(50, 800);
        Prev_Button.Size = new Size(100, 35);
        Prev_Button.FlatStyle = FlatStyle.Popup;
        // Prev_Button.Click += new System.EventHandler(Prev_Button_Click);
        this.Controls.Add( Prev_Button );

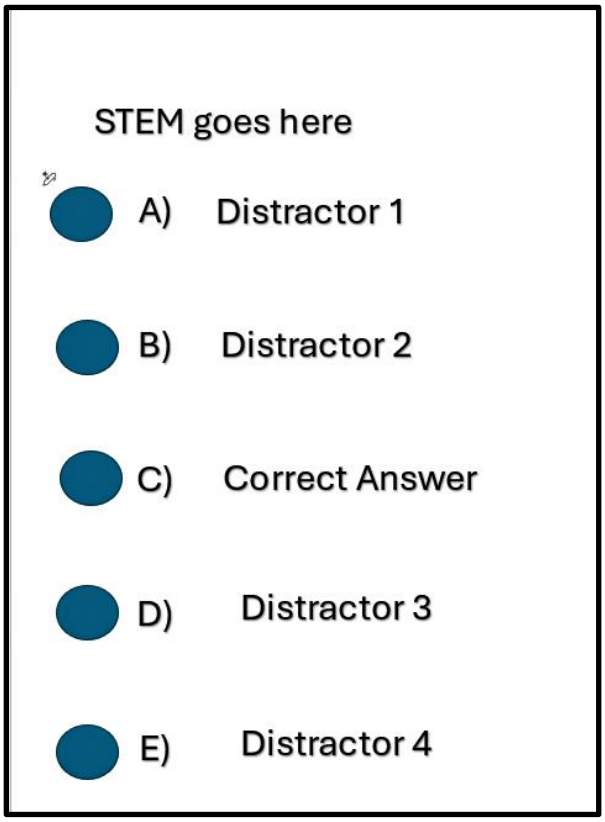
        Next_Button.Text = "Next";
        Next_Button.Location = new Point(250, 800);
        Next_Button.Size = new Size(100, 35);
        Next_Button.FlatStyle = FlatStyle.Popup;
        // Next_Button.Click += new System.EventHandler(Next_Button_Click);
        this.Controls.Add(Next_Button);

        Submit_Button.Text = "Submit";
        Submit_Button.Location = new Point(450, 800);
        Submit_Button.Size = new Size(100, 35);
        Submit_Button.FlatStyle = FlatStyle.Popup;
        // Submit_Button.Click += new System.EventHandler(Submit_Button_Click);
        this.Controls.Add(Submit_Button);

        Refresh_Button.Text = "Refresh";
        Refresh_Button.Location = new Point(650, 800);
        Refresh_Button.Size = new Size(100, 35);
        Refresh_Button.FlatStyle = FlatStyle.Popup;
        // Refresh_Button.Click += new System.EventHandler(Refresh_Button_Click);
        this.Controls.Add(Refresh_Button);
    }
    1 reference
    private void timer1_Tick(object sender, EventArgs e)
```

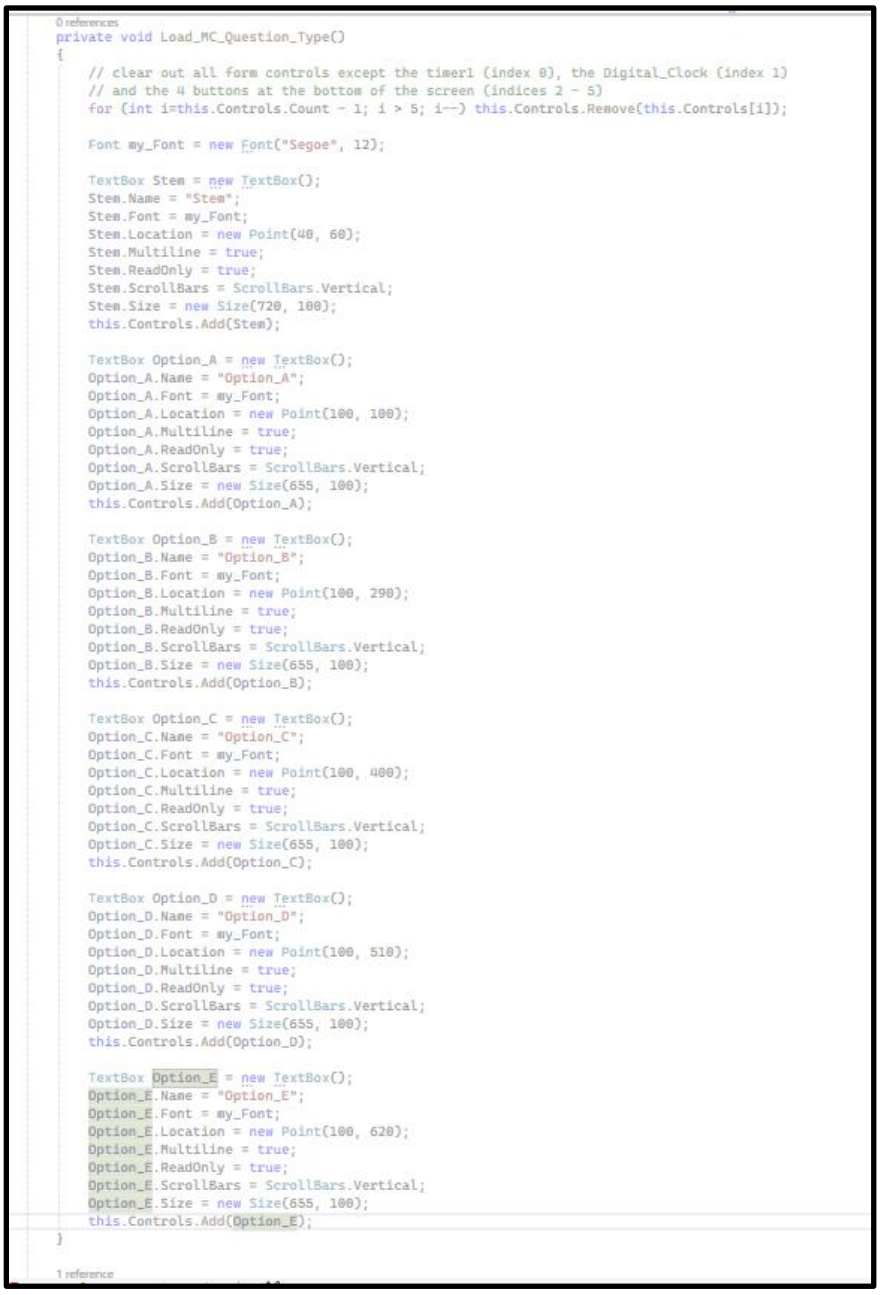
Okay, let's move onto building a multiple-choice question display. My rough-cut design is shown here. (Figure 16A.8) Psychometricians get paid a lot of money, so to justify their salaries they give their multiple-choice question components special names. More specifically, it is not the question it is the stem. Those answer options which are not correct are called distractors. Some questions will have only a single distractor, i.e. true/false question, while other questions can have 3 or even 4 distractors. We will build a GUI which can accommodate any of these configurations.

Figure 16A.8: Multiple-Choice Question Display



The first part of my Load_MC_Question_Type routine looked like this. (Figure 16A.9) Start by deleting all controls on the form from the previous question which may have been of a different type and then recreate a bunch of TextBoxes. Please note that we are creating all the objects necessary for a 4-distractor multiple-choice question. If some of these are not being used by a particular question, we will simply make them invisible at the time we load the data associated with a specific item.

Figure 16A.9: Load_MC_Question_Type Part 1



```
0 references
private void Load_MC_Question_Type()
{
    // clear out all form controls except the timer1 (index 0), the Digital_Clock (index 1)
    // and the 4 buttons at the bottom of the screen (indices 2 - 5)
    for (int i=this.Controls.Count - 1; i > 5; i--) this.Controls.Remove(this.Controls[i]);

    Font my_Font = new Font("Segoe", 12);

    TextBox Stem = new TextBox();
    Stem.Name = "Stem";
    Stem.Font = my_Font;
    Stem.Location = new Point(40, 60);
    Stem.Multiline = true;
    Stem.ReadOnly = true;
    Stem.ScrollBars = ScrollBars.Vertical;
    Stem.Size = new Size(720, 100);
    this.Controls.Add(Stem);

    TextBox Option_A = new TextBox();
    Option_A.Name = "Option_A";
    Option_A.Font = my_Font;
    Option_A.Location = new Point(100, 100);
    Option_A.Multiline = true;
    Option_A.ReadOnly = true;
    Option_A.ScrollBars = ScrollBars.Vertical;
    Option_A.Size = new Size(655, 100);
    this.Controls.Add(Option_A);

    TextBox Option_B = new TextBox();
    Option_B.Name = "Option_B";
    Option_B.Font = my_Font;
    Option_B.Location = new Point(100, 290);
    Option_B.Multiline = true;
    Option_B.ReadOnly = true;
    Option_B.ScrollBars = ScrollBars.Vertical;
    Option_B.Size = new Size(655, 100);
    this.Controls.Add(Option_B);

    TextBox Option_C = new TextBox();
    Option_C.Name = "Option_C";
    Option_C.Font = my_Font;
    Option_C.Location = new Point(100, 480);
    Option_C.Multiline = true;
    Option_C.ReadOnly = true;
    Option_C.ScrollBars = ScrollBars.Vertical;
    Option_C.Size = new Size(655, 100);
    this.Controls.Add(Option_C);

    TextBox Option_D = new TextBox();
    Option_D.Name = "Option_D";
    Option_D.Font = my_Font;
    Option_D.Location = new Point(100, 510);
    Option_D.Multiline = true;
    Option_D.ReadOnly = true;
    Option_D.ScrollBars = ScrollBars.Vertical;
    Option_D.Size = new Size(655, 100);
    this.Controls.Add(Option_D);

    TextBox Option_E = new TextBox();
    Option_E.Name = "Option_E";
    Option_E.Font = my_Font;
    Option_E.Location = new Point(100, 620);
    Option_E.Multiline = true;
    Option_E.ReadOnly = true;
    Option_E.ScrollBars = ScrollBars.Vertical;
    Option_E.Size = new Size(655, 100);
    this.Controls.Add(Option_E);
}

1 reference
```

I next needed to add those RadioButtons to the left of the answer options. The catch here is these guys need to be mutually exclusive. The answer comes in the form of a GroupBox. A GroupBox can have a collection of contained objects while remaining invisible.

Here is how you do it. You first instantiate a GroupBox and add it to the Form's Controls collection. Locate that guy at 0,0 and make its size greater than the objects to be contained within it. Now go ahead and create your RadioButtons and add them to the GroupBox's Controls collection. Here is my shot at the code which by the way is in that same Load_MC_Question_Type routine. (Figure 16A.10).

Figure 16A.10: Load_MC_Question_Type Part 2

```
    Option_E.ScrollBars = ScrollBars.Vertical;
    Option_E.Size = new Size(655, 100);
    this.Controls.Add(Option_E);

    Font my_font2 = new Font("Segoe",14,FontStyle.Bold);

    GroupBox myGroupBox = new GroupBox();
    myGroupBox.Location = new Point(0, 0);
    myGroupBox.Size = new Size(100, 700);
    this.Controls.Add(myGroupBox);

    RadioButton Radio_Button_A = new RadioButton();
    Radio_Button_A.Name = "Radio_Button_A";
    Radio_Button_A.Text = "A)";
    Radio_Button_A.Font = my_font2;
    Radio_Button_A.Location = new Point(40, 200);
    myGroupBox.Controls.Add(Radio_Button_A);

    RadioButton Radio_Button_B = new RadioButton();
    Radio_Button_B.Name = "Radio_Button_B";
    Radio_Button_B.Text = "B)";
    Radio_Button_B.Font = my_font2;
    Radio_Button_B.Location = new Point(40, 310);
    myGroupBox.Controls.Add(Radio_Button_B);

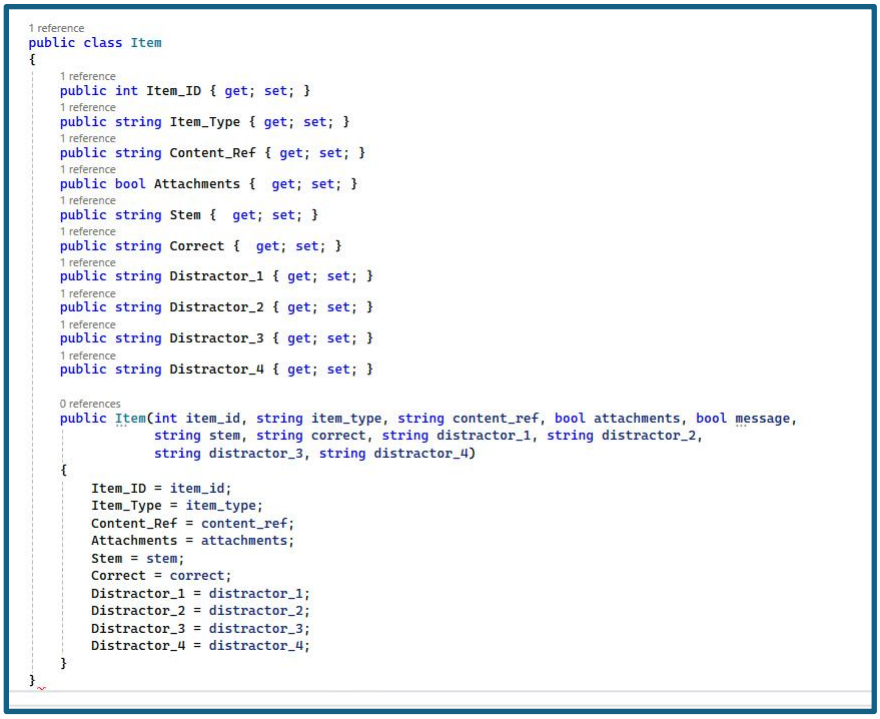
    RadioButton Radio_Button_C = new RadioButton();
    Radio_Button_C.Name = "Radio_Button_C";
    Radio_Button_C.Text = "C)";
    Radio_Button_C.Font = my_font2;
    Radio_Button_C.Location = new Point(40, 420);
    myGroupBox.Controls.Add(Radio_Button_C);

    RadioButton Radio_Button_D = new RadioButton();
    Radio_Button_D.Name = "Radio_Button_D";
    Radio_Button_D.Text = "D)";
    Radio_Button_D.Font = my_font2;
    Radio_Button_D.Location = new Point(40, 530);
    myGroupBox.Controls.Add(Radio_Button_D);

    RadioButton Radio_Button_E = new RadioButton();
    Radio_Button_E.Name = "Radio_Button_E";
    Radio_Button_E.Text = "E)";
    Radio_Button_E.Font = my_font2;
    Radio_Button_E.Location = new Point(40, 640);
    myGroupBox.Controls.Add(Radio_Button_E);
}
```

At this point I needed an Item class so that was my next class definition. My Item class is shown here. Again, please notice the parallel parameterized constructor. (Figure 16A.11)

Figure 16A.11 Item Class Definition



```
1 reference
public class Item
{
    1 reference
    public int Item_ID { get; set; }
    1 reference
    public string Item_Type { get; set; }
    1 reference
    public string Content_Ref { get; set; }
    1 reference
    public bool Attachments {  get; set; }
    1 reference
    public string Stem {  get; set; }
    1 reference
    public string Correct {  get; set; }
    1 reference
    public string Distractor_1 { get; set; }
    1 reference
    public string Distractor_2 { get; set; }
    1 reference
    public string Distractor_3 { get; set; }
    1 reference
    public string Distractor_4 { get; set; }

    0 references
    public Item(int item_id, string item_type, string content_ref, bool attachments, bool message,
                string stem, string correct, string distractor_1, string distractor_2,
                string distractor_3, string distractor_4)
    {
        Item_ID = item_id;
        Item_Type = item_type;
        Content_Ref = content_ref;
        Attachments = attachments;
        Stem = stem;
        Correct = correct;
        Distractor_1 = distractor_1;
        Distractor_2 = distractor_2;
        Distractor_3 = distractor_3;
        Distractor_4 = distractor_4;
    }
}
```

I next instantiated a number of items in my form load routine so we had something with which to work. An array of user-defined classes was also needed to emulate the set of questions in a test. I aptly named my array my_items. (Figure 16A.12)

Figure 16A.12: Item Instantiation

```
1 reference
private void Form1_Load(object sender, EventArgs e)
{
    this.Name = "Content";
    this.Size = new System.Drawing.Size(820, 900);
    this.Text = "Content";
    Digital_Clock.Size = new Size(150, 40);
    Digital_Clock.Location = new Point(670, 20);
    Digital_Clock.Text = "00:00";
    timer1.Interval = 1000;

    up_or_down = "Up";
    stakes = "Low";
    type = "MC";
    duration_in_secs = 0;  // not timed
    passing_point = 80;

    setup_timer();
    timer1.Tick += new System.EventHandler(timer1_Tick);
    Load_Bottom_Buttons();
    Load_MC_Question_Type();

    Item my_Item_1 = new Item(1, "MC", "", false, false, "In what year did the United States gain indepence?",
                              "1776", "1566", "2000", "1851", "1976");
    Item my_Item_2 = new Item(2, "MC", "", false, false, "Are whales mammals?",
                              "Yes", "No", "", "", "");
    Item my_Item_3 = new Item(3, "MC", "", false, false, "Horse hooves are equivalent to what part of the human body?",
                              "The fingernail and last toe bone", "Metatarsils", "Hair", "Nose", "");
    Item my_Item_4 = new Item(4, "MC", "", false, false, "What is the square root of -1?",
                              "i", "e", "-1", "undefined", "");
    Item my_Item_5 = new Item(5, "MC", "", false, false, "Is the C# programming language case sensitive?",
                              "True", "False", "", "", "");
    Item[] my_items = new Item[] { my_Item_1, my_Item_2, my_Item_3, my_Item_4, my_Item_5 };
    Globals.my_items_global = my_items;

    // MC_test_driver(my_items);
}
```

Let’s call it a wrap at this point. We’ll start loading items to the screen in our next lesson with that MC_test_driver routine we commented out in the last code.

So, what did we learn in this lesson?

- GroupBox class
- RadioButton class
- TextBox class
- Adding objects to a form in code

In our next lesson we hook some code onto those buttons at the bottom of the screen and create a routine which randomizes and displays multiple-choice items.